Design and Analysis of Algorithms
Taubakabyl Nurlybek
Assignment 2 - Data Structures

Deadline: 04.10.2026 23:59

Submission: ZIP archive named DAA_Assignment2_name_surname_group.zip
- Upload to Moodle.
- Include a link to your GitHub repository (branch main, tag v1.0).
- **Defense:** Practice classes in Week 5. You will be asked to run your tests and benchmark, explain your code, justify your loop invariants, and answer questions about trade-offs.
- **AI tools:** Allowed only for debugging and explaining concepts. You must be able to explain every line you submit.
- **Cheating is strongly prohibited and will be graded as 0!**

Scenario: In-Memory Workload Engine

In Assignment 1 your engine processed large arrays in batch mode. Now it must keep data in memory and serve three different workloads: fast access by index, frequent insertions and removals at the boundaries, and priority-based job scheduling.

Standard collections are convenient, but your team needs its own transparent implementations: you have to count the exact number of operations, prove that they are correct, and find out why two structures with the same Big-O can differ several times in real running time.

In **Assignment 2**, you will:

- Implement **DynamicArray**, **MyLinkedList** and **MinHeap** from scratch.
- Prove the correctness of two operations with **loop invariants**.
- Measure all structures on **4 workloads** and explain the results.

Learning Objectives

- Implement basic **data structures** from scratch, without java.util collections.
- Prove that a loop is correct with a **loop invariant** (initialization, maintenance, termination).
- Give tight **O, Ω and Θ** bounds for the best, average and worst case of every operation.
- Count **physical operations** (steps, moves, comparisons) and export them to CSV.
- Explain performance differences caused by **CPU cache locality** and **pointer chasing**.
- Keep a clean **Git history** with feature branches and a release tag.

System Requirements

1. Data Structures

Implement three structures from scratch. Do **not** use java.util.ArrayList, java.util.LinkedList or java.util.PriorityQueue:

- **Element Type:** All structures store primitive int values (int[] inside DynamicArray and MinHeap). Do not use generics or Integer: boxed objects break the cache comparison in your report.
- **DynamicArray:** add(x) (amortized O(1), grow the array 2x when it is full), add(index, x), remove(index), get(index), contains(x).
- **MyLinkedList:** Singly or doubly linked list with the same five methods. Use the name MyLinkedList so it does not clash with java.util.LinkedList.
- **MinHeap:** Array-based binary heap: insert(x) (bubble-up), peekMin() in O(1), extractMin() (bubble-down).
- **Invalid Input:** get, add(index, x) and remove throw IndexOutOfBoundsException; peekMin and extractMin on an empty heap throw IllegalStateException.

2. Loop Invariants

Choose **two** operations with a loop from your own code (for example contains, remove(index) or bubble-down). For each one write in the report:

- **Invariant:** the exact statement that is true before every iteration.
- **Initialization:** the invariant is true before the first iteration.
- **Maintenance:** if it is true before an iteration, it is still true before the next one.
- **Termination:** when the loop stops, the invariant gives the property you need.
- **Conclusion:** 1-2 sentences on why this proves the whole operation is correct.

3. Workloads & Benchmark

Run 4 workloads for sizes n = 100, 1 000, 10 000, 100 000. Fill every structure with the **same** data, generated with new Random(42):

- **W1 - Random Access:** DynamicArray and MyLinkedList. Fill with n values, then perform **10 000** get(index) calls with a random index.
- **W2 - Search:** DynamicArray and MyLinkedList. Perform **1 000** contains(x) queries, where half of the values are present in the structure and half are not.
- **W3 - Insert & Remove:** DynamicArray and MyLinkedList, two variants:
  - **head** - 1 000 insertions and 1 000 removals at index 0;
  - **middle** - 1 000 insertions and 1 000 removals at index n / 2.
- **W4 - Priority Processing:** MinHeap. Insert n values, then call extractMin() n times and check that the returned values are in non-decreasing order.

4. Metrics & CSV

- **What to Count:** steps - one read of an array cell **or** one move to the next node; moves - one element shifted inside the array **or** one link (pointer) update in the list; comparisons - one comparison of two elements.
- **Why It Matters:** get(i) costs 1 step in DynamicArray but about i steps in MyLinkedList, and a shift moves elements only in the array. Count operations inside the methods, do not estimate them afterwards.
- **Repeat Runs:** Run every case **5 times** and save the **median** time. Discard the first (warm-up) runs: the JVM needs time to compile your code.
- **CSV:** Save results.csv with columns workload,variant,structure,n,time_ms,steps,moves,comparisons. Use variant head or middle for W3 and - for the other workloads.

5. Tests (JUnit 5)

- **Correctness:** Compare your structures with an expected result on random data. You may use java.util classes **in tests only**.
- **Edge Cases:** Empty structure, one element, duplicate values, first and last index, invalid index.
- **Heap Property:** After every insert and extractMin the condition a[parent] <= a[child] must hold.
- **Sorted Output:** n calls of extractMin() must return the values in non-decreasing order.

6. Report (REPORT.md or PDF, 3-5 pages)

- **Complexity Table:** Best, average and worst case for every operation of all three structures, plus auxiliary space. Use Θ when the bound is tight, O or Ω when it is not, and add one line of justification per row.
- **Loop Invariant Proofs:** The two proofs from part 2.
- **Plots:** One chart per workload:
  - **Time vs n** - both structures on the same chart;
  - **Steps / moves / comparisons vs n**.
- **Discussion (10-15 sentences):**
  - Why DynamicArray is faster for get(i) and even for simple iteration (CPU cache lines, spatial locality);
  - Why the list can be slower even when both structures perform the same number of steps (pointer chasing, object headers, garbage collector);
  - When MyLinkedList or MinHeap is the better choice.

7. Git Workflow

- **Branches:** main (only working code, tag v1.0) and feature/array, feature/list, feature/heap, feature/metrics.
- **Commits:** Small steps with clear messages, e.g. feat(array): add dynamic resizing, test(heap): check heap property, docs(report): add plots.
- **Layout:** src/main/java (structures, metrics, benchmark), src/test/java (tests), results/results.csv, results/plots/, README.md, REPORT.md.

Design Quality Standards

Best-Case Design (Target)

- **Same Interface:** DynamicArray and MyLinkedList have the same methods, so one benchmark runs both.
- **Honest Counters:** Steps, moves and comparisons are counted inside the operations.
- **Reproducible Results:** The same seed and one command produce results.csv.
- **Readable Plots:** Axis labels, units and a legend on every chart.

Worst-Case Design (Avoid)

- **Hidden Collections:** java.util.ArrayList used inside your "own" structure.
- **Boxed Values:** Storing Integer instead of int, which destroys the cache comparison.
- **Zero Counters:** MyLinkedList reporting 0 operations in W3 because only array shifts are counted.
- **One Run Only:** A single measurement without warm-up, where n = 100 looks slower than n = 1 000.

Bonus Points (Extra +15%)

Task A: Memory Footprint (+10%) Use **JOL (Java Object Layout)** to measure how many bytes n elements really take in DynamicArray, MyLinkedList and MinHeap. Draw a Memory (MB) vs n plot and explain the overhead of node objects (object header, pointers, alignment).

Task B: O(n) buildHeap (+5%) Implement Floyd's bottom-up buildHeap(int[] array) that builds a heap in O(n). Compare its time and comparison count with n separate insert(x) calls (O(n log n)) and explain the difference in the report.

Deliverables

1. **Source Code:** Maven or Gradle project with the three structures, metrics and JUnit 5 tests.
2. **results.csv:** All workloads with the columns listed above.
3. **Plots:** PNG charts in results/plots/.
4. **Report:** REPORT.md or PDF with the complexity table, proofs, plots and discussion.
5. **README:** How to build the project, run the tests and run the benchmark.

Grading Rubric

Criterion	Weight	What is evaluated	High-Performance Indicators
Data Structures	25%	DynamicArray, MyLinkedList, MinHeap	Written from scratch; int storage; 2x growth; heap property holds after every operation.

Criterion	Weight	What is evaluated	High-Performance Indicators
Loop Invariants	15%	Two proofs	All five parts present; clear link between the invariant and correctness.
Asymptotic Analysis	15%	Complexity table (O / Ω / Θ)	Best, average and worst case for every operation; short justification per row.
Benchmark & Metrics	20%	Workloads 1-4, results.csv	Correct counters; median of 5 runs after warm-up; complete CSV with all variants.
Report, Plots & Discussion	15%	Cache and memory analysis	Readable charts; explains why equal Big-O can give different real time.
Testing & Git	10%	JUnit 5 tests, README, history	Edge cases covered; feature branches; tag v1.0; project builds with one command.